

BMIT3173 Integrative Programming

ASSIGNMENT 202605

Student Name : Wong Siew Lam

Student ID : 25WMR09791

Programme : Bachelor in Information Technology (Honours)
(Information Security)

Tutorial Group : Group 4

System Title : LearnSync - Integrated Educational Resource and
Collaborative Learning Portal

Chosen SDG : SDG 4 - Quality Education

Modules : Module 4 - Skill Assessment and Quiz

AI Tools Usage Policy & Disclosure

This assignment is governed by the **TAR UMT Policy for the Use of Artificial Intelligence (AI) (PO/111:26)** and is classified under the **YELLOW (Limited AI)** category. AI tools may be used only for the purposes listed below, not to produce the technical work, analysis, or original content being assessed.

Mandatory submission: Every student must complete and submit the **AI Usage Disclosure Form** (below) with the report, whether or not AI tools were used, by 6th September 2026. A report submitted without the Form is incomplete and may not be marked.

Permitted (Yellow) uses of AI tools:

- Language refinement (grammar, spelling, clarity) of text you wrote, without altering its technical meaning.
- Clarifying taught concepts (e.g. how a design pattern, web-service protocol, or attack works) to aid your understanding.
- Debugging help on code you wrote, where AI only locates or explains an error you then fix yourself.
- Brainstorming early ideas (e.g. SDG framings or topic angles), provided the chosen direction and its justification are your own.

Prohibited uses of AI tools:

- Generating the PHP code, design-pattern implementation, class diagrams, or web-service code being assessed.
- Producing core arguments or analysis, including the design-pattern justification, threat analysis, and secure-coding rationale.
- Uploading confidential, proprietary, or others' personal data to public AI tools (Sections 2.5 and 5 of the AI Policy).

You remain responsible for the accuracy and originality of your work; AI errors do not excuse incorrect content. The lecturer may ask you to explain or demonstrate any part to verify authorship. Undisclosed or prohibited AI use is a breach of academic integrity under Section 4 of the AI Policy. **Complete the Form below; if AI was used, list each use and cite the tool(s) in your References (APA 7th).**

AI Usage Disclosure Form

Declaration (tick one):

- ☐ No AI tools were used in the preparation of this report.
- ☒ AI tools were used as declared in the table below.

AI Tool Used (name & version)	Purpose / How It Was Used	Report Section(s) Affected
Grammarly (2026 Edition)	Used solely for language refinement, spelling verification, and grammatical sentence flow of my own written text without altering technical logic.	Sections 1, 2, 4.1, 4.3, 5.1

If no AI tools were used, tick "No AI tools used". Non-disclosure breaches the AI Policy.

I declare this Form is true and complete and that my AI use complied with the AI Policy and the Yellow conditions above.

Sign: Lam

Date: 6/9/2026

Table of Contents

AI Tools Usage Policy & Disclosure	2
1. Introduction to the System	4
1.1 System Overview	4
1.2 The Sustainable Development Goal	4
1.3 System Contribution to SDG 4 & Scope	5
2. Module Description	6
2.1 Scope of Module 4: Skill Assessment and Quiz	6
2.2 Functional Breakdown & Class Paths	6
F4.1: Building a Quiz and Its Questions	6
F4.2: The Three Question Types	8
F4.3: Sitting the Paper	9
F4.4: Marking, and Recording What Was Answered	10
F4.5: Reviewing a Finished Attempt	13
3. Entity Classes	14
3.1 Entity Class Diagram	14
3.2 Entity Class Implementation (Eloquent ORM Mapping)	14
4. Design Pattern	17
4.1 Description of Design Pattern	17
4.2 Implementation of Design Pattern	19
1. The Strategy Interface (app/Patterns/Strategy/GradingStrategy.php)	19
2. Concrete Strategy One, Equality (app/Patterns/Strategy/MCQGradingStrategy.php)	19
3. Concrete Strategy Two, Set Comparison (app/Patterns/Strategy/MultipleAnswerGradingStrategy.php)	21
4. Choosing the Strategy (app/Patterns/Strategy/GradingStrategyResolver.php)	23
5. The Context, Which Never Asks What Type a Question Is (app/Http/Controllers/QuizAttemptController.php)	24
4.3 Justification of Design Pattern	26
5. Software Security	27
5.1 Potential Threat/Attack	27
5.2 Secure Coding Practice	29
6. Web Services	33
6.1 Web Service Exposure	33
6.2 Web Service Consumption	37
7. References	41
8. Appendices	42
Appendix A: Automated Testing Results	42
Appendix B: GitHub Repository URL	45

1. Introduction to the System

1.1 System Overview

LearnSync is a web based Learning Management System (LMS). Lecturers use it to upload notes, set quizzes, give assignments and mark student work. Students use it to study the notes, sit the quizzes, hand in assignments, watch their progress, and ask questions in a course forum.

The problem LearnSync solves is about proof. In most systems, the certificate a student earns is just a PDF file. Anyone can open that PDF in a word processor and change the marks. If an employer wants to check whether a certificate is real, they must email the college and wait days for a reply. Most employers do not bother.

LearnSync fixes this. Every certificate it creates carries a unique ID, a QR code anyone can scan, and a hidden security code proving the record has not been changed. Anyone can check a certificate in seconds, for free, without an account.

1.2 The Sustainable Development Goal

LearnSync is built around United Nations Sustainable Development Goal 4: Quality Education (SDG 4).

What SDG 4 aims to do:

SDG 4 wants to make sure education is fair, good quality and available to everyone by 2030 (United Nations, 2015). Target 4.3 is about giving everyone fair access to affordable college and university education. Target 4.4 is about increasing the number of people who have real skills for getting a job.

1.3 System Contribution to SDG 4 & Scope

LearnSync supports SDG 4 in three ways:

1. Who benefits: Students who earn certificates, lecturers who teach and mark the courses, administrators who control access, and outside people such as employers who need to check a qualification.
2. Marking that is instant and identical for everyone: Module 4 marks a quiz the moment it is submitted, so a student finds out straight away rather than waiting a week. More importantly, the marking is done by code, so every student's paper is judged by exactly the same rule. A tired lecturer marking a hundred papers at midnight is not the same marker they were that morning, and code does not get tired. Fair assessment is a large part of what SDG 4 means by quality.
3. Making qualifications checkable: Every certificate carries a unique ID, a QR code and a security code, so an employer can confirm it instantly and for free.

What the system does not do: LearnSync does not replace official government accreditation, and quizzes are not proctored. There is no camera monitoring and no lockdown browser, so a quiz here measures understanding rather than proving who sat at the keyboard.

2. Module Description

2.1 Scope of Module 4: Skill Assessment and Quiz

I am the developer in charge of Module 4, the Skill Assessment and Quiz Module. I designed and built:

- the screens a lecturer uses to build a quiz and write its questions and answer options
- three different question types, each needing a completely different way of marking
- the paper a student sits, including the live counter for multiple answer questions
- the marking engine that runs the moment a paper is submitted
- the record of what each student actually answered, question by question, so an attempt can be reviewed afterwards
- passing the final mark to Module 5, which owns the grades table

The three question types are the reason this module uses the Strategy pattern. Marking a tick box question and marking a typed answer are not variations of one job. They are genuinely different algorithms.

2.2 Functional Breakdown & Class Paths

F4.1: Building a Quiz and Its Questions

- Description: A lecturer creates a quiz on a course they own, giving it a title and a time limit in minutes. They then add questions one at a time, choosing the question type and typing the answer options. For a tick box question they mark which options are correct, and for a typed answer they write the model answer.
- A lecturer can only build a quiz on their own course. The check is made on the permission key first and the course ownership second, so a lecturer holding the permission is still refused on somebody else's course.
- Class Paths:
 - Controller: `app/Http/Controllers/QuizController.php` (create, store, show, destroy, storeQuestion, destroyQuestion)
 - Models: `app/Models/Quiz.php`, `app/Models/Question.php`, `app/Models/Answer.php`
 - View Templates: `resources/views/quizzes/create.blade.php`, `resources/views/quizzes/show.blade.php`

ADD A QUESTION

Type

Multiple choice (one answer)

Multiple choice (one answer)

Multiple choice (several answers)

Fill in the blank

Options

Select the radio button beside the correct one.

☒ Option 1

☐ Option 2

☐ Option 3

☐ Option 4

☐ Option 5

Add question

Figure 2.1: The Quiz Builder Showing the Question Type Dropdown

Web Services and Integration Basics

3 questions · 10 minute limit

QUESTIONS

Which HTTP method retrieves a resource without modifying it?

mcq · Exact match against the option marked correct.

✓ GET

· POST

· DELETE

· PATCH

Remove

Which architectural pattern separates data, presentation and control logic, and underpins Laravel?

text · Case- and spacing-insensitive similarity against the model answer, tolerating typos.

✓ MVC

✓ Model View Controller

✓ Model-View-Controller

Remove

Which of these are valid HTTP methods? Select exactly 2.

multi · Set comparison: every correct option must be selected, with partial credit for a near miss.

✓ GET

✓ POST

· SEND

· FETCH

Remove

Figure 2.2: A Quiz with Its Questions and Answer Options Listed

F4.2: The Three Question Types

- Description: Each type is marked by a completely different algorithm, which is what Section 4 is about.

Type	What the student does	How it is marked
One answer (mcq)	Picks a single option	Is the chosen option the one flagged correct?
Several answers (multi)	Ticks more than one option	Compares two sets of options, with partial credit
Fill in the blank (text)	Types an answer	Measures how similar the typed text is to the model answer

For a several answers question, how many to tick is worked out from how many options are flagged correct, rather than being typed in by the lecturer. A question claiming to want three answers while holding only two correct options would be impossible to answer, so the number is derived instead of stored.

- Class Paths:
 - Interface: `app/Patterns/Strategy/GradingStrategy.php`
 - Strategies: `app/Patterns/Strategy/MCQGradingStrategy.php`, `MultipleAnswerGradingStrategy.php`, `TextMatchGradingStrategy.php`
 - Resolver: `app/Patterns/Strategy/GradingStrategyResolver.php`

ADD A QUESTION

Type

Multiple choice (one answer) ▼

Multiple choice (one answer)

Multiple choice (several answers)

Fill in the blank

Options

Select the radio button beside the correct one.

☒ Option 1

☐ Option 2

☐ Option 3

☐ Option 4

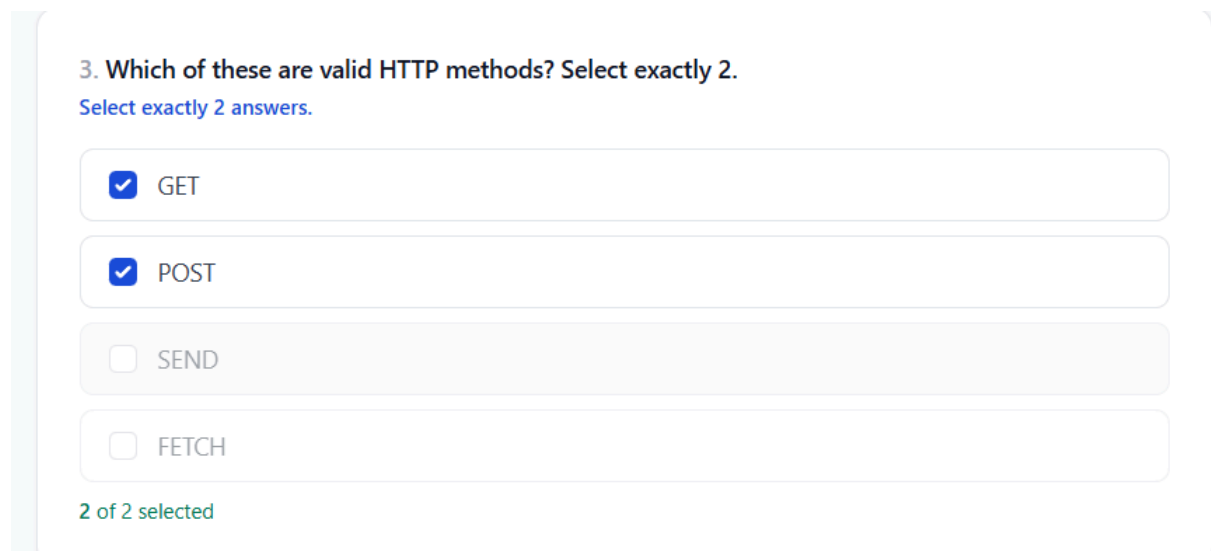
☐ Option 5

Add question

Figure 2.3: The Three Question Types Offered in the Builder

F4.3: Sitting the Paper

- Description: A student opens the quiz and answers each question. A several answers question shows how many options are wanted, keeps a live count of how many have been ticked, and stops accepting further clicks once the limit is reached, so nobody has to guess what is expected.
- Only enrolled students may sit a quiz. A lecturer cannot take their own quiz, because the specification says lecturers do not sit assessments, and this is checked as the quiz.attempt permission rather than by comparing roles.
- The correct answers are never sent to the browser. The page renders each option's text and its database ID, and nothing else. Section 5 covers why that matters.
- Class Paths:
 - Controller: `app/Http/Controllers/QuizAttemptController.php` (create, store, show)
 - View Template: `resources/views/quizzes/attempt.blade.php`



3. Which of these are valid HTTP methods? Select exactly 2.

Select exactly 2 answers.

☒ GET

☒ POST

☐ SEND

☐ FETCH

2 of 2 selected

Figure 2.4: The Quiz Paper Showing a Several Answers Question with Its Live Counter

F4.4: Marking, and Recording What Was Answered

- Description: Marking happens the instant the paper is submitted. The controller walks the questions, asks the resolver for the right strategy for each one, and calls grade(). It never asks what type a question is.
- Every answer is saved to quiz_attempt_answers with the student's response, whether it was correct, and the marks awarded. Without this table there would be nowhere to record what a student actually wrote, so an attempt could be neither graded properly nor reviewed afterwards.
- The whole thing runs inside one database transaction, so a paper is either fully marked and recorded, or not recorded at all. A half marked attempt would show a student a score that did not match their answers.
- The final percentage is written as a Grade, which is Module 5's table. That single write is what wakes the credentialing chain in Module 1.
- Class Paths:
 - Controller: app/Http/Controllers/QuizAttemptController.php (store)
 - Models: app/Models/QuizAttempt.php, app/Models/QuizAttemptAnswer.php
 - View Template: resources/views/quizzes/result.blade.php

[← Back to Web Services and Integration Basics](#)

YOUR SCORE

33.33%

F

1 of 3 correct · 31s taken

Which HTTP method retrieves a resource without modifying it? INCORRECT

Your answer: DELETE

Expected: GET

Marked 0 / 1

Which architectural pattern separates data, presentation and control logic, and underpins Laravel? INCORRECT

Your answer: ok

Expected: MVC / Model View Controller / Model-View-Controller

Marked 0 / 1

Which of these are valid HTTP methods? Select exactly 2. CORRECT

Your answer: GET POST

Marked 1 / 1

Figure 2.5: The Result Page Immediately After Submitting

The image shows a screenshot of a quiz review interface. It contains four question cards, each with a question, the user's answer, the expected answer, and a status indicator. The first card is green and marked 'CORRECT'. The other three cards are red and marked 'INCORRECT'.

Question	Your answer	Expected	Status
Which project phase produces the requirements specification?	Analysis		CORRECT
What is the name for a scheduled point at which progress is formally reviewed?	hihi	milestone / milestones	INCORRECT
Which document defines a project's objectives, scope and deliverables at the outset?	Gantt chart	Project charter	INCORRECT
What name is given to uncontrolled growth in a project's requirements after work has begun?	hihi	scope creep / requirement creep	INCORRECT

Figure 2.6: An Attempt Reviewed Question by Question, Showing What Was Answered

F4.5: Reviewing a Finished Attempt

- Description: A student can reopen their own finished attempt and see each question, what they answered, whether it was right, and the marks given. A lecturer can open any attempt on their own course.
- A student cannot open anybody else's attempt. The check compares the attempt's owner against the person asking and refuses with 403 otherwise, because the specification states that a student cannot view another student's results.
- Class Paths:
 - Controller: `app/Http/Controllers/QuizAttemptController.php` (show)
 - View Template: `resources/views/quizzes/result.blade.php`

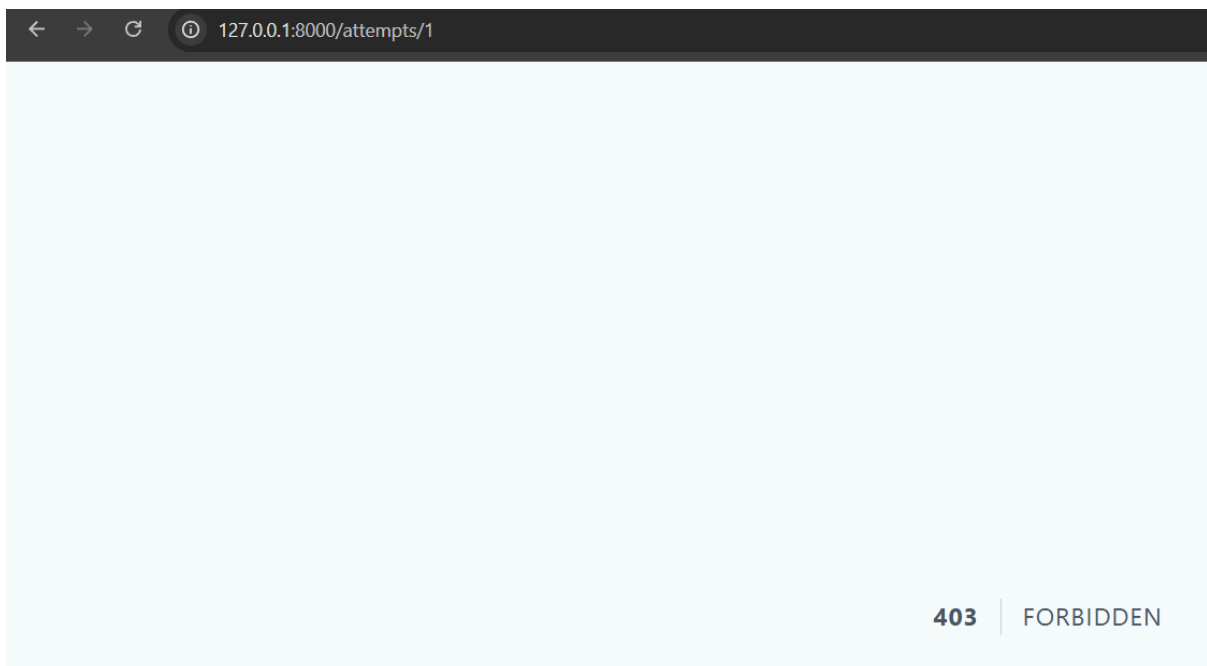


Figure 2.7: A Student Receiving 403 When Opening Another Student's Attempt

3. Entity Classes

3.1 Entity Class Diagram

The diagram below shows the entity classes using object references, meaning one object points to another object, instead of database foreign keys.

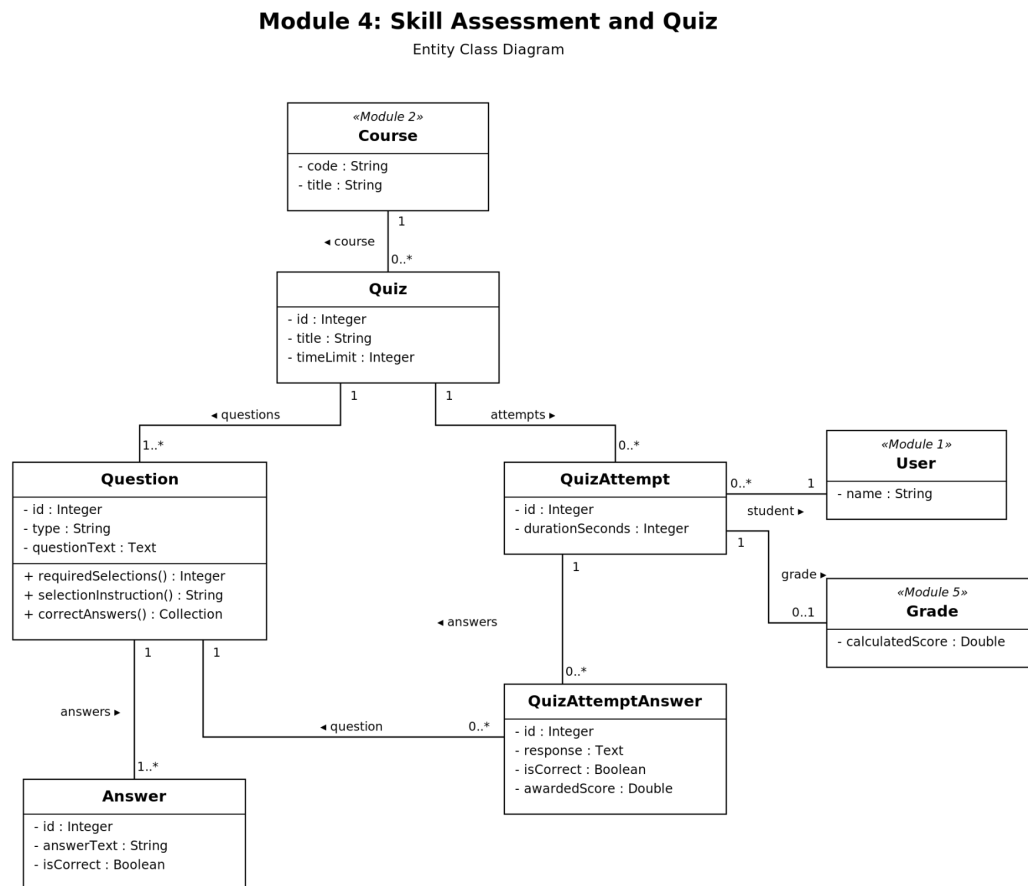


Figure 3.1: Entity Class Diagram for Module 4 Skill Assessment and Quiz

3.2 Entity Class Implementation (Eloquent ORM Mapping)

The classes are written in PHP using Laravel's Eloquent ORM. Relationships are written as methods, so the code never writes SQL. Calling `$question->answers` returns a collection of Answer objects, so the marking code can ask each one whether it is correct by following the link between objects.

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Question extends Model
{
    protected $fillable = ['quiz_id', 'type', 'question_text'];

    // The three types, named as constants so the resolver and the forms
    // cannot drift apart by using different strings.
    public const TYPE_MCQ = 'mcq';
    public const TYPE_MULTI = 'multi';
    public const TYPE_TEXT = 'text';

    /** Object reference: a question belongs to one quiz */
    public function quiz(): BelongsTo
    {
        return $this->belongsTo(Quiz::class);
    }

    /** Object reference: a question offers many answer options */
    public function answers(): HasMany
    {
        return $this->hasMany(Answer::class);
    }

    /**
     * How many options must be ticked, worked out from how many are
     * flagged correct rather than stored. A question claiming to want
     * three answers while holding two correct ones would be unanswerable.
     */
    public function requiredAnswerCount(): int
    {
        return $this->answers->where('is_correct', true)->count();
    }
}
```

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\Relations\HasOne;

class QuizAttempt extends Model
{
    protected $fillable = ['quiz_id', 'student_id', 'duration_seconds'];

    /** Object reference: an attempt is at one quiz */
    public function quiz(): BelongsTo
    {
        return $this->belongsTo(Quiz::class);
    }

    /** Object reference: an attempt belongs to one student */
    public function student(): BelongsTo
    {
        return $this->belongsTo(User::class, 'student_id');
    }

    /** Object reference: an attempt records one answer per question */
    public function answers(): HasMany
    {
        return $this->hasMany(QuizAttemptAnswer::class);
    }

    /** Object reference: Module 5 owns the resulting mark */
    public function grade(): HasOne
    {
        return $this->hasOne(Grade::class);
    }
}
```

4. Design Pattern

4.1 Description of Design Pattern

For Module 4, I used the Strategy Design Pattern, which is a Gang of Four Behavioural Pattern.

What the pattern means:

Gamma, Helm, Johnson and Vlissides (1994) describe the Strategy Pattern as one that defines a family of algorithms, puts each one in its own class, and makes them interchangeable. Strategy lets the algorithm change independently of the code that uses it.

In simple words, sometimes there are several different ways of doing the same job, and which one to use depends on the situation. The obvious approach is a long if statement, or a switch, that checks the situation and then runs the right block of code. That works, but the checking code and all the algorithms end up mixed together in one method. Every new case means editing that method, and the method grows until nobody wants to touch it.

The Strategy pattern gives each algorithm its own class, all offering the same method name. The calling code holds one of those objects and calls the method. It never checks which one it is holding.

How this works in my module:

Marking a quiz question is exactly this situation. The three types are not variations of one calculation. They are three unrelated algorithms:

- One answer: compare the chosen option's ID against the ID of the option flagged correct. A simple equality test.
- Several answers: compare two sets of IDs, checking whether the set chosen matches the set required, and awarding partial credit in proportion to the overlap.
- Fill in the blank: compare two pieces of text and measure how similar they are, so that a small spelling mistake still passes.

There is no single formula covering all three. Set comparison and string similarity have nothing in common.

The parts of the pattern in my code:

Pattern Role	Class in my module
Strategy (the shared interface)	GradingStrategy
Concrete Strategy	MCQGradingStrategy
Concrete Strategy	MultipleAnswerGradingStrategy
Concrete Strategy	TextMatchGradingStrategy
Context (chooses and uses one)	QuizAttemptController, through GradingStrategyResolver
Result object	GradedAnswer, carrying the verdict, the mark and an explanation

How Strategy differs from a similar pattern:

Strategy is sometimes confused with State, which Module 5 uses. Both hold an object and call a method on it instead of using an if statement. The difference is who decides which object. With Strategy, the caller picks the algorithm and it does not change during the job: a multiple choice question is marked by the multiple choice strategy from beginning to end. With State, the object changes itself as the work progresses: a submission moves from draft to submitted to graded on its own.

Diagram link:

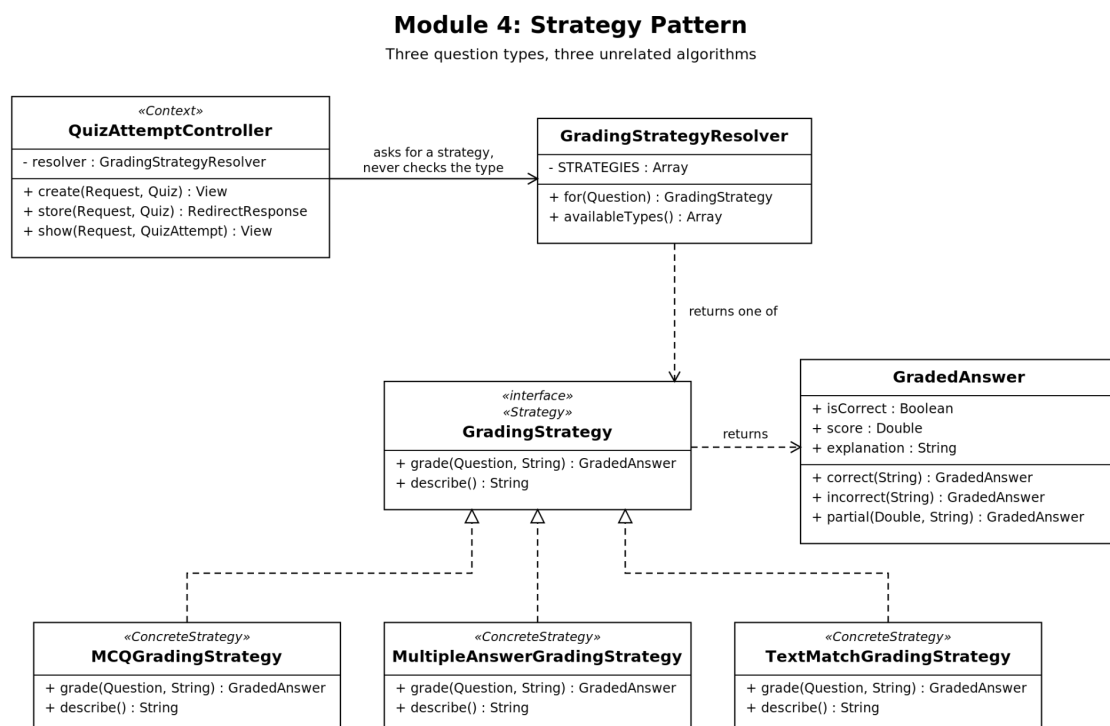


Figure 4.1: Strategy Pattern Class Diagram for Quiz Marking

4.2 Implementation of Design Pattern

1. The Strategy Interface (app/Patterns/Strategy/GradingStrategy.php)

```

namespace App\Patterns\Strategy;

use App\Models\Question;

/**
 * The strategy interface.
 *
 * Different question types are marked by genuinely different algorithms. An
 * MCQ is an exact match against the chosen option, while a fill in the blank
 * has to tolerate case, spacing and small typing errors.
 *
 * Putting each algorithm behind this interface means the controller never
 * grows a switch statement over question types. It asks the resolver for a
 * strategy and calls grade(), and a new question type later means a new
 * class rather than an edit to existing code.
 */
interface GradingStrategy
{
    /**
     * Mark one response.
     *
     * @param Question $question the question being answered
     * @param string|null $response what the student gave
     * @return GradedAnswer the verdict and the mark out of 1
     */
    public function grade(Question $question, ?string $response): GradedAnswer;

    /** A short description of how this strategy marks, shown to lecturers. */
    public function describe(): string;
}

```

2. Concrete Strategy One, Equality (app/Patterns/Strategy/MCQGradingStrategy.php)

```

namespace App\Patterns\Strategy;

use App\Models\Question;

/**
 * CONCRETE STRATEGY -- a single correct option.
 *
 * The simplest of the three: the response is one Answer id, and it is either
 * the id flagged correct or it is not.
 */
class MCQGradingStrategy implements GradingStrategy
{
    public function grade(Question $question, ?string $response): GradedAnswer
    {
        if (blank($response)) {
            return GradedAnswer::incorrect('No option selected.');
```

3. Concrete Strategy Two, Set Comparison

(app/Patterns/Strategy/MultipleAnswerGradingStrategy.php)

```

namespace App\Patterns\Strategy;

use App\Models\Question;

/**
 * CONCRETE STRATEGY -- multiple correct answers.
 *
 * A third algorithm again, genuinely unlike the other two: the single choice
 * strategy tests membership of one id, the text strategy measures string
 * similarity, and this one compares two sets.
 */
class MultipleAnswerGradingStrategy implements GradingStrategy
{
    public function grade(Question $question, ?string $response): GradedAnswer
    {
        $required = $question->answers->where('is_correct', true)->pluck('id')
            ->map(fn ($id) => (string) $id)->sort()->values();

        if ($required->isEmpty()) {
            return GradedAnswer::partial(1.0, 'This question has no correct options set; full
marks awarded.');
```

```

        }

        $chosen = collect(explode(',', (string) $response))
            ->map(fn ($id) => trim($id))->filter()->unique()->values();

        if ($chosen->isEmpty()) {
            return GradedAnswer::incorrect('No options selected.');
```

```

        }

        $expected = $required->count();

        // SECURITY (Module 4): the form enforces the count in the browser,
        // so a mismatch here means the form was bypassed. Grade it honestly
        // rather than trusting that the browser did its job.
        if ($chosen->count() !== $expected) {
            return GradedAnswer::incorrect(
                "Select exactly {$expected} answers, you selected {$chosen->count()}."
            );
        }

        $correctlyChosen = $chosen->intersect($required)->count();

        if ($correctlyChosen === $expected) {
            return GradedAnswer::correct('All ' . $expected . ' correct options selected.');
```

```

        }

        /**
         * Partial credit in proportion to how many were right. Getting three

```

```
* of four is genuinely better than getting none, and all or nothing
* would make a four answer question far harsher than four separate
* questions would be.
*/
return GradedAnswer::partial(
    $correctlyChosen / $expected,
    "{$correctlyChosen} of {$expected} correct options selected."
);
}

public function describe(): string
{
    return 'Set comparison: every correct option must be selected, with partial credit for a
near miss.';
}
}
```

4. Choosing the Strategy (app/Patterns/Strategy/GradingStrategyResolver.php)

```

namespace App\Patterns\Strategy;

use App\Models\Question;
use InvalidArgumentException;

/**
 * Picks the strategy for a question type.
 *
 * This is the ONE place that maps a type string to an algorithm, which is
 * what keeps the swap dynamic: the controller calls for($question) and has
 * no idea which class comes back.
 */
class GradingStrategyResolver
{
    /** @var array<string, class-string<GradingStrategy>> */
    private const STRATEGIES = [
        Question::TYPE_MCQ => MCQGradingStrategy::class,
        Question::TYPE_TEXT => TextMatchGradingStrategy::class,
        Question::TYPE_MULTI => MultipleAnswerGradingStrategy::class,
    ];

    public function for(Question $question): GradingStrategy
    {
        $class = self::STRATEGIES[$question->type] ?? null;

        if ($class === null) {
            throw new InvalidArgumentException(
                "No grading strategy is registered for question type \"{$question->type}\"."
            );
        }

        return new $class();
    }

    /**
     * The question types the engine can mark, for building form dropdowns.
     * The builder form reads this, so a type can never appear in the
     * dropdown without a strategy existing to mark it.
     */
    public static function availableTypes(): array
    {
        return [
            Question::TYPE_MCQ => 'Multiple choice (one answer)',
            Question::TYPE_MULTI => 'Multiple choice (several answers)',
            Question::TYPE_TEXT => 'Fill in the blank',
        ];
    }
}

```

5. The Context, Which Never Asks What Type a Question Is

(app/Http/Controllers/QuizAttemptController.php)

```
class QuizAttemptController extends Controller
{
    // The resolver is injected, so the controller does not even construct it
    public function __construct(private GradingStrategyResolver $resolver)
    {
    }

    public function store(Request $request, Quiz $quiz): RedirectResponse
    {
        $this->authoriseStudent($request, $quiz);
        $quiz->load('questions.answers');

        // ... validation ...

        // One transaction, so a paper is either fully marked and recorded
        // or not recorded at all. A half marked attempt would show a score
        // that did not match the answers.
        $attempt = DB::transaction(function () use ($request, $quiz, $responses, $data) {

            $attempt = QuizAttempt::create([
                'quiz_id' => $quiz->id,
                'student_id' => $request->user()->id,
                'duration_seconds' => (int) ($data['duration_seconds'] ?? 0),
            ]);

            $earned = 0.0;

            foreach ($quiz->questions as $question) {
                // THE STRATEGY IS CHOSEN PER QUESTION, AT RUN TIME.
                // Note there is no if, no switch and no mention of mcq,
                // multi or text anywhere in this method.
                $strategy = $this->resolver->for($question);
                $result = $strategy->grade($question, $responses[$question->id] ?? null);

                $attempt->answers()->create([
                    'question_id' => $question->id,
                    'response' => $responses[$question->id] ?? null,
                    'is_correct' => $result->isCorrect,
                    'awarded_score' => $result->score,
                ]);

                $earned += $result->score;
            }

            $percentage = round(($earned / max(1, $quiz->questions->count())) * 100, 2);

            // Module 5's authoritative record. Writing it is what wakes the
            // credentialing chain in Module 1.
            Grade::create([
```

```
        'quiz_attempt_id' => $attempt->id,  
        'calculated_score' => $percentage,  
    ]);  
  
    return $attempt;  
});  
  
return redirect()->route('attempts.show', $attempt)  
    ->with('success', 'Quiz submitted and marked.');
```

4.3 Justification of Design Pattern

1. The three algorithms are genuinely unrelated, so one method could not have held them. Comparing one ID to another, comparing two sets with partial credit, and measuring the similarity of two pieces of text share no logic at all. Written as one method with a switch, the marking code would have been three unrelated blocks stitched together, and a change to the text matching rules would have meant editing the same method that marks tick boxes. Separate classes keep unrelated things apart.
2. The claim was tested by real growth, and it held. The module first shipped with two question types, one answer and fill in the blank. The several answers type was added afterwards. Adding it meant writing one new class and adding one line to the resolver's array. QuizAttemptController was not touched at all. That is the Open Closed Principle, and I can point at the commit rather than only asserting it.
3. Each algorithm can be tested on its own. MCQGradingStrategy needs a question and a response and returns a GradedAnswer. It touches no HTTP request, no session and no controller, so a test can construct one directly and check its output. If the marking lived inside the controller, testing the text matching rules would need a whole simulated web request with a logged in student.
4. It keeps the marking rules readable to a lecturer. Each strategy carries a describe() method saying in one sentence how it marks, such as set comparison with partial credit. The builder screen shows these, so a lecturer choosing a question type can see what the marking will do without reading any code.
5. It follows the Single Responsibility Principle. QuizAttemptController has one job, which is running an attempt: check the student is allowed, walk the questions, save the answers, write the mark. Deciding whether a particular answer is right is a different job with different reasons to change, and it lives in its own class.

5. Software Security

5.1 Potential Threat/Attack

Threat 1: Answer Disclosure and Client Side Marking Tampering (OWASP A04: Insecure Design)

Attack Description:

A quiz is a piece of software that decides whether somebody passes. That makes it a target in a way most pages are not, because a successful attack here does not steal data, it manufactures a qualification.

There are two ways to attack it, and both come from the same root cause, which is trusting the browser.

The first is answer disclosure. A quiz page must send the question and the answer options to the browser so the student can read them. The obvious but disastrous mistake is to send the `is_correct` flag along with each option, either as part of the HTML or in a piece of JavaScript holding the data. A student who presses F12 and reads the page source then sees exactly which options are correct, before answering anything. They do not need any tools beyond the ones already built into their browser, and there is no trace in any log, because reading a page that was sent to you is not suspicious.

The second is client side marking tampering. If the browser worked out the score and posted the result, a student could skip the quiz entirely and post `score=100` directly, using the browser's own developer tools or a tool such as Postman. The server would record a perfect mark for a paper that was never answered. Because a quiz mark automatically triggers the certificate machinery in Module 1, this would end in a genuine, verifiable certificate awarded for nothing.

Risk Impact:

Certificates issued for assessments that were never actually passed. Because those certificates carry a real credential ID and pass the public verification check, the damage reaches outside the system to employers who trust it.

Threat 2: Session Hijacking During an Authenticated Quiz Session (OWASP A07: Identification and Authentication Failures)**Attack Description:**

Every request a student makes while sitting a quiz is authenticated by a session cookie. That cookie is what tells LearnSync who is answering. Anybody holding a copy of it can submit answers as that student.

There are several ways an attacker gets one. On the shared campus wireless network, an unencrypted HTTP connection lets anyone running a packet sniffer read the cookie straight off the wire. On a shared laboratory computer, a student who walks away without logging out leaves a live session behind for the next person. A cross site scripting flaw anywhere in the application would let JavaScript read the cookie and send it away, which is why Module 3's output encoding matters to my module too.

There is also session fixation, which works the other way round. Instead of stealing a session, the attacker supplies one. They obtain a session identifier, trick the victim into using it through a crafted link, and wait for the victim to log in. If the application keeps the same session identifier after login rather than issuing a fresh one, the attacker's copy is now an authenticated session belonging to the victim.

For a quiz this is worse than for an ordinary page, because sitting a paper takes many minutes. The window during which a valid session exists is long, and the attacker only needs to submit once.

Risk Impact:

An attacker submits a quiz as another student, either sabotaging their mark or, if they are helping them cheat, completing the paper for them. A hijacked lecturer session is worse still, because a lecturer can create and delete quizzes and read every student's results.

5.2 Secure Coding Practice

Secure Practice 1: The Server Marks the Paper, and the Correct Answers Never Leave It

OWASP Category: Access Control and Trust Boundaries. The defence against both halves of Threat 1 is a single rule applied without exception: the browser is never told which answers are correct, and never asked to work out a score.

The quiz paper renders only the option text and the option's database ID:

```

{{-- resources/views/quizzes/attempt.blade.php --}}

@foreach ($question->answers as $answer)
    <label>
        {{-- SECURITY (Module 4): the VALUE is the answer's database id, and
            the LABEL is its text. The is_correct flag is never rendered,
            never placed in a data attribute, and never sent to any script.
            Pressing F12 reveals nothing but the options themselves. --}}
        <input type="radio" name="responses[{{ $question->id }}" value="{{ $answer->id }}">
        <span>{{ $answer->answer_text }}</span>
    </label>
@endforeach

```

What the student's browser receives is the same information a printed exam paper would give them: the question and the choices. The answer key stays on the server, in the answers.is_correct column, and no route in the application returns it to a student.

The marking then happens entirely on the server, driven by the Strategy pattern:

```

// app/Http/Controllers/QuizAttemptController.php

// SECURITY (Module 4): the browser posts only WHICH OPTIONS WERE CHOSEN.
// It never posts a score, and if it did, the value would be ignored,
// because the score below is calculated here from the stored answer key.
foreach ($quiz->questions as $question) {
    $strategy = $this->resolver->for($question);
    $result = $strategy->grade($question, $responses[$question->id] ?? null);

    $attempt->answers()->create([
        'question_id' => $question->id,
        'response'    => $responses[$question->id] ?? null,
        'is_correct'  => $result->isCorrect,    // decided here, on the server
        'awarded_score' => $result->score,      // calculated here, on the server
    ]);

    $earned += $result->score;
}

$percentage = round(($earned / max(1, $quiz->questions->count())) * 100, 2);

```

Why this defeats tampering completely. The request contains no score for an attacker to change. Posting extra fields achieves nothing, because the controller reads only responses, and the percentage is computed from \$earned, which is built up from the strategies' own return values. The worst an attacker can do by crafting a request is submit answers they could have submitted through the form anyway.

The same principle is applied where the browser does help the student, in the live counter on a several answers question. That counter is a convenience, not a control, and the server treats it as such:

```
// app/Patterns/Strategy/MultipleAnswerGradingStrategy.php

// SECURITY (Module 4): the form enforces the count in the browser, so a
// mismatch here means the form was bypassed. Grade it honestly rather
// than trusting that the browser did its job.
if ($chosen->count() !== $expected) {
    return GradedAnswer::incorrect(
        "Select exactly {$expected} answers, you selected {$chosen->count()}."
    );
}
```

Bypassing the browser check is possible and gains the attacker nothing. The server checks the count again and marks the answer wrong. This is the correct shape for any client side helper: useful when it works, ignored when it is absent.

Secure Practice 2: Session Management, with a Fresh Identifier on Login and Cookies JavaScript Cannot Read

OWASP Category: Session Management. Four settings, working together.

```
// config/session.php

// (a) Sessions are stored in the DATABASE, not in a file or the cookie
//     itself. The cookie holds only a random identifier, so nothing about
//     the user travels over the network and nothing can be read or edited
//     by tampering with the cookie's contents.
'driver' => env('SESSION_DRIVER', 'database'),

// (b) A session expires after two hours of inactivity, which bounds how
//     long an abandoned session on a shared laboratory machine stays
//     usable.
'lifetime' => (int) env('SESSION_LIFETIME', 120),

// (c) httpOnly means JavaScript cannot read the cookie through
//     document.cookie. If a cross site scripting flaw ever appeared
//     anywhere in the application, it still could not steal the session.
'http_only' => env('SESSION_HTTP_ONLY', true),

// (d) sameSite lax means the browser will not attach this cookie to
//     requests coming from another website, which blocks the cross site
//     request forgery route into any quiz action.
'same_site' => env('SESSION_SAME_SITE', 'lax'),
```

Against session fixation specifically, Laravel's authentication regenerates the session identifier at the moment of login, so any identifier an attacker planted beforehand becomes worthless the instant the victim signs in:

```
// app/Http/Controllers/Auth/AuthenticatedSessionController.php

public function store(LoginRequest $request): RedirectResponse
{
    $request->authenticate();

    // SECURITY (Module 4 relies on this): a NEW session id is issued on
    // login. Any id an attacker planted beforehand is discarded here, so
    // it never becomes an authenticated session.
    $request->session()->regenerate();

    return redirect()->intended(route('dashboard', absolute: false));
}
```

And the account controls Module 1 provides close the loop, because a session belonging to a deactivated or locked account is rejected on the very next request rather than continuing until it expires:

```
// bootstrap/app.php

// Runs on every web request. A deactivated or locked account is signed out
// rather than merely being shown empty pages.
$middleware->web(append: [
    EnsureAccountIsActive::class,
    EnsurePasswordIsChanged::class,
]);
```

Why these belong together. Storing sessions in the database means the cookie is only a pointer, so there is nothing in it worth stealing except the pointer itself. `httpOnly` stops scripts reading that pointer, `sameSite` stops other sites using it, the `lifetime` limits how long a forgotten one survives, and `regeneration on login` stops an attacker supplying one in advance. Each closes a different route to the same attack.

6. Web Services

Module 4 works with web services in both directions, and both are built and working in the code.

As a provider, it exposes getQuizResult. Module 3 uses it to write a notification saying a quiz has been marked, and Module 5 uses it when gathering figures for a cohort. Neither of them reads my tables or repeats my marking rules.

As a consumer, it calls Module 2's getCOURSEInfo, so that a quiz can be labelled with its course code and title. Module 2 owns course data, so Module 4 asks rather than querying it directly.

I chose REST with JSON over SOAP because it is lighter, needs no WSDL contract file or XML wrapper, and can be called straight from a browser, from Postman, or from PHP.

Both halves follow one shared Interface Agreement held in app/Support/Ifa.php. Every request carries a requestID and a timeStamp. Every response carries a status of S, F or E, a timeStamp, and the requestID echoed back.

6.1 Web Service Exposure

Interface Agreement (IFA) for Service Exposure

	Description
Protocol	RESTful Web Service (JSON over HTTP)
Function Description	Returns a student's best attempt at one quiz, with the score, letter grade and pass status
Source Module	Module 4: Skill Assessment and Quiz Module
Target Module	Module 3 (Student Forum and Notifications), Module 5 (Academic Progress Analytics)
HTTP Method & URL	GET /api/quizzes/result
Controller Action	App\Http\Controllers\Api\QuizApiController@result
Function Name	getQuizResult
Authentication	Shared key in an X-API-Key header

Web Services Request Parameters (IFA Requirement)

Field Name	Field Type	Mandatory/Optional	Description	Format
quizId	Integer	Mandatory	The quiz to look up.	Whole number above 0
studentId	Integer	Mandatory	Whose result	Whole number

			is wanted.	above 0
requestID	String	Mandatory	A unique ID so the request can be traced.	Letters, numbers, hyphens. Max 64
timeStamp	String	Mandatory	The time the request was made.	YYYY-MM-DDTHH:MM:SSZ

Web Services Response Parameters (IFA Requirement)

Field Name	Field Type	Mandatory / Optional	Description	Format
status	String	Mandatory	Whether the request worked.	S for Success, F for Fail, E for Error
timeStamp	String	Mandatory	The time the answer was created.	YYYY-MM-DDTHH:MM:SSZ
data.requestID	String	Mandatory	The request ID sent back, for matching.	Letters, numbers, hyphens
data.attempted	Boolean	Mandatory	Whether the student has sat this quiz.	true or false
data.graded	Boolean	Optional	Whether a mark exists yet.	true or false
data.attemptCount	Integer	Optional	How many times they sat it.	0 or above
data.bestScore	Double	Optional	The best mark across all attempts.	0.00 to 100.00
data.letterGrade	String	Optional	The letter for that mark.	A, A-, B+, B and so on
data.passed	Boolean	Optional	Whether the best mark is a pass.	true or false

Code Implementation (app/Http/Controllers/Api/QuizApiController.php)

```

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Grade;
use App\Models\QuizAttempt;
use App\Support\GradeScale;
use App\Support\Ifa;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;
use Throwable;

/**
 * MODULE 4 exposes: getQuizResult.
 *
 * Returns the student's BEST attempt, not their latest. A re-sit that went
 * badly should not overwrite a good earlier result when another module asks
 * how this student did.
 */
class QuizApiController extends Controller
{
    public function result(Request $request): JsonResponse
    {
        // Ifa::baseRules() supplies the two fields every service demands,
        // so all five members validate them the same way.
        $validator = validator($request->all(), Ifa::baseRules() + [
            'quizId' => ['required', 'integer', 'min:1'],
            'studentId' => ['required', 'integer', 'min:1'],
        ]);

        if ($validator->fails()) {
            return Ifa::fail($request, [
                'attempted' => false,
                'errors' => $validator->errors()->all(),
            ]);
        }

        try {
            $attemptIds = QuizAttempt::where('quiz_id', $request->integer('quizId'))
                ->where('student_id', $request->integer('studentId'))
                ->pluck('id');

            if ($attemptIds->isEmpty()) {
                // Not an error. The student simply has not sat this quiz.
                return Ifa::success($request, [
                    'attempted' => false,
                    'attemptCount' => 0,
                ]);
            }

            $grades = Grade::whereIn('quiz_attempt_id', $attemptIds)->get();

```

```

        if ($grades->isEmpty()) {
            return Ifa::success($request, [
                'attempted' => true,
                'attemptCount' => $attemptIds->count(),
                'graded' => false,
            ]);
        }

        $best = (float) $grades->max('calculated_score');

        return Ifa::success($request, [
            'attempted' => true,
            'graded' => true,
            'attemptCount' => $attemptIds->count(),
            'bestScore' => round($best, 2),
            'letterGrade' => GradeScale::letterFor($best),
            'passed' => GradeScale::isPass($best),
        ]);

    } catch (Throwable $e) {
        Log::error('getQuizResult failed', ['error' => $e->getMessage()]);

        // The exception message goes to the log, never to the caller.
        return Ifa::error($request, ['attempted' => false]);
    }
}
}

```

Real response from the running system:

```

{
  "status": "S",
  "timeStamp": "2026-09-05T16:30:35Z",
  "data": {
    "requestID": "QUZ-REQ-55667",
    "attempted": true,
    "graded": true,
    "attemptCount": 1,
    "bestScore": 75,
    "letterGrade": "A-",
    "passed": true
  }
}

```

6.2 Web Service Consumption

Interface Agreement (IFA) for Service Consumption

	Description
Protocol	RESTful Web Service (JSON over HTTP)
Function Description	Returns the course code, title, student count and optionally the lecturer, for a given course ID
Source Module	Module 2: Academic Resources Repository Module
Consuming Module	Module 4: Skill Assessment and Quiz Module
HTTP Method & URL	GET /api/courses/info
Client Class	App\Support\Api\CourseInfoClient@fetch
Function Name	getCourseInfo
Protocol	RESTful Web Service (JSON over HTTP)

Web Services Request Parameters (IFA Requirement for Consumption)

Field Name	Field Type	Mandatory/Optional	Description	Format
courseId	Integer	Mandatory	The course to look up.	Whole number above 0
queryFlag	Integer	Mandatory	How much detail is needed.	1: code and title 2: also the lecturer
requestID	String	Mandatory	A tracking ID made by Module 4, prefixed CRS-REQ.	Letters, numbers, hyphens
timeStamp	String	Mandatory	The time the request was sent.	YYYY-MM-DDTHH:MM:SSZ

Web Services Response Parameters (IFA Requirement for Consumption)

Field Name	Field Type	Mandatory/ Optional	Description	Format
status	String	Mandatory	Whether the request worked.	S for Success, F for Fail, E for Error
timeStamp	String	Mandatory	The time the answer was created.	YYYY-MM-DDTHH:MM:SSZ
data.courseCode	String	Mandatory	The public course code.	Letters and numbers, e.g. BMIT3173
data.courseTitle	String	Mandatory	The full course name.	Letters and numbers
data.studentCount	Integer	Mandatory	How many students are enrolled.	0 or above
data.instructorName	String	Optional	The lecturer's name.	Only sent when queryFlag is 2

Consumption Code Implementation (app/Support/Api/CourseInfoClient.php)

```

namespace App\Support\Api;

/**
 * CONSUMES Module 2's getCourseInfo service.
 *
 * Module 2 owns course data, so Module 4 asks Module 2's service rather
 * than reading its `courses` table. The ownership boundary holds even
 * across an HTTP call.
 */
class CourseInfoClient extends ServiceClient
{
    // Stamped on this client's request IDs, so a call in Module 2's log
    // can be traced back to its caller.
    protected function requestPrefix(): string
    {
        return 'CRS-REQ';
    }

    public function fetch(int $courseId): ?array
    {
        return $this->get('/courses/info', [
            'courseId' => $courseId,
            'queryFlag' => 1,
        ]);
    }

    public function fetchWithInstructor(int $courseId): ?array
    {
        return $this->get('/courses/info', [
            'courseId' => $courseId,
            'queryFlag' => 2,
        ]);
    }
}

```

The shared sending logic lives in the parent class, so every consuming module handles a failure the same way:

```

// app/Support/Api/ServiceClient.php (extract)

$body = $response->json();

/**
 * The IFA status is the contract, not the HTTP code. A provider can return
 * 200 with a status of F, meaning it understood the question perfectly well
 * and could not answer it.
 */
if (!Ifa::succeeded($body)) {
    return null;
}

```

```
return $body['data'] ?? null;
```

Every client returns null when a call does not succeed, and never throws. If Module 2 is unavailable, a quiz page still loads and simply shows the course code it already holds locally, rather than failing.

Real response from the running system:

```
{
  "status": "S",
  "timeStamp": "2026-09-05T16:30:17Z",
  "data": {
    "requestID": "CRS-REQ-11223",
    "courseCode": "BMIT3173",
    "courseTitle": "Integrative Programming",
    "studentCount": 33
  }
}
```

Figure 6.3: Postman Showing Module 4 Consuming Module 2's Course Info Service

7. References

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley Professional.

Laravel LLC. (2026). Laravel 12.x documentation: Eloquent ORM, validation and session configuration. <https://laravel.com/docs/12.x>

OWASP Foundation. (2021). OWASP Top 10:2021 The ten most critical web application security risks. Open Web Application Security Project. <https://owasp.org/Top10/>

OWASP Foundation. (2022). OWASP secure coding practices quick reference guide (Version 2.1). <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>

United Nations. (2015). Transforming our world: The 2030 agenda for sustainable development (Goal 4: Quality Education). United Nations Department of Economic and Social Affairs. <https://sdgs.un.org/goals/goal4>

8. Appendices

Appendix A: Automated Testing Results

```
c:\xampp\htdocs\edusystem>php artisan test

PASS Tests\Unit\ExampleTest
✓ that true is true 0.01s

PASS Tests\Feature\Auth\AuthenticationTest
✓ login screen can be rendered 0.59s
✓ users can authenticate using the login screen 0.08s
✓ users can not authenticate with invalid password 0.24s
✓ users can logout 0.03s

PASS Tests\Feature\Auth\EmailVerificationTest
✓ email verification screen can be rendered 0.04s
✓ email can be verified 0.04s
✓ email is not verified with invalid hash 0.03s

PASS Tests\Feature\Auth\PasswordConfirmationTest
✓ confirm password screen can be rendered 0.04s
✓ password can be confirmed 0.03s
✓ password is not confirmed with invalid password 0.23s

PASS Tests\Feature\Auth\PasswordResetTest
✓ reset password link screen can be rendered 0.03s
✓ reset password link can be requested 0.25s
✓ reset password screen can be rendered 0.26s
✓ password can be reset with valid token 0.28s

PASS Tests\Feature\Auth\PasswordUpdateTest
✓ password can be updated 0.04s
✓ correct password must be provided to update password 0.03s

PASS Tests\Feature\Auth\RegistrationTest
✓ open registration route does not exist 0.04s
✓ an unknown token is rejected 0.03s
✓ an expired invitation is refused 1.66s
✓ an already accepted invitation is refused 0.56s
✓ a valid invitation creates the account with its fixed role 0.05s
✓ a token cannot be redeemed twice 0.60s

PASS Tests\Feature\AwardAndActivityNotificationTest
✓ earning a certificate notifies the holder 0.53s
✓ earning a badge notifies the student 0.05s
✓ marking submitted work notifies the student 0.04s
✓ a marked quiz does not notify because the result is already on screen 0.03s
```

```

✓ inviting a student to a course notifies them 0.03s
✓ a switched off preference still suppresses the new types 0.03s

PASS Tests\Feature\AwardRuleTest
✓ an admin defined average score rule awards a badge 0.06s
✓ an admin defined quizzes completed rule counts distinct quizzes 0.05s
✓ an admin defined certificate rule mints a real credential 0.41s
✓ a certificate rule does not mint twice 0.56s
✓ a certificate rule is never handed out as a badge 0.50s
✓ deactivating a rule stops it without removing awards already made 0.06s

PASS Tests\Feature\CalendarEventDetailTest
✓ an enrolled student can open an events detail page 0.07s
✓ a student cannot open an event for a course they are not in 0.04s
✓ a lecturer cannot open an event for another lecturers course 0.04s
✓ an institution wide event is visible to everyone 0.05s
✓ a meeting shows a join button 0.05s
✓ an event with no link shows no join button 0.06s
✓ a malformed meeting link does not crash or render a button 0.06s
✓ a javascript url is never rendered as a join button 0.05s
✓ a student does not see the names of their classmates 0.06s

PASS Tests\Feature\CourseEnrolmentTest
✓ a student cannot leave a course themselves 0.05s
✓ the owning lecturer can remove a student 0.04s
✓ a lecturer cannot remove a student from another lecturers course 0.05s
✓ a student cannot remove a classmate 0.04s
✓ removing somebody who is not enrolled is a 404 0.04s

PASS Tests\Feature\CourseMaterialUploadTest
✓ a lecturer can upload a normal document 0.06s
✓ a php file is refused 0.04s
✓ a php file renamed to look like a pdf is still refused 0.05s
✓ an html file is refused 0.04s
✓ a javascript url cannot be saved as an external material 0.04s
✓ an ordinary external link is accepted 0.04s
✓ a lecturer cannot upload to another lecturers course 0.04s

PASS Tests\Feature\ExampleTest
✓ the application returns a successful response 0.04s

PASS Tests\Feature\ProfileTest
✓ profile page is displayed 0.06s
✓ profile information can be updated 0.04s
✓ email verification status is unchanged when the email address is unchanged 0.04s
✓ user can delete their account 0.03s
✓ correct password must be provided to delete account 0.03s

PASS Tests\Feature\ProfileTest
✓ profile page is displayed 0.06s
✓ profile information can be updated 0.04s
✓ email verification status is unchanged when the email address is unchanged 0.04s
✓ user can delete their account 0.03s
✓ correct password must be provided to delete account 0.03s

PASS Tests\Feature\SubjectExpertBadgeTest
✓ passing every quiz in the subject awards the badge 0.06s
✓ attempting without passing does not award it 0.04s
✓ a subject with no quizzes awards nothing 0.03s
✓ resubmitting does not award a second copy 0.05s
✓ a quiz added afterwards does not revoke the badge 0.05s
✓ the badge is scoped to its own subject 0.06s

PASS Tests\Feature\WebServiceTest
✓ every response carries status timestamp and the request id 0.05s
✓ a request without the mandatory ifa fields is refused 0.03s
✓ an internal service refuses a caller with no api key 0.03s
✓ an internal service refuses a wrong api key 0.03s
✓ the credential service is public and needs no key 0.43s
✓ detail flag one does not disclose the holder 0.45s
✓ detail flag two returns the holder 0.54s
✓ an unknown credential reports not found rather than erroring 0.03s
✓ a tampered credential is reported over the service 0.45s
✓ the quiz service returns the best attempt 0.06s
✓ the analytics service returns cohort figures only 0.05s
✓ the notification service writes to the inbox 0.03s
✓ the notification service refuses a type outside the allow list 0.03s
✓ module 1 consumes module 2s course info service 0.08s
✓ module 5 consumes module 1s credential service 0.42s
✓ module 3 consumes module 4s quiz result service 0.05s
✓ a client returns null rather than throwing when the service is unreachable 1.07s

Tests: 86 passed (200 assertions)
Duration: 13.37s

```

Figure 8.1: Terminal Output of php artisan test Showing 86 Passing Tests

```
c:\xampp\htdocs\edusystem>php artisan test --filter=SubjectExpertBadgeTest

PASS Tests\Feature\SubjectExpertBadgeTest
✓ passing every quiz in the subject awards the badge 0.58s
✓ attempting without passing does not award it 0.04s
✓ a subject with no quizzes awards nothing 0.03s
✓ resubmitting does not award a second copy 0.05s
✓ a quiz added afterwards does not revoke the badge 0.04s
✓ the badge is scoped to its own subject 0.05s

Tests: 6 passed (10 assertions)
Duration: 0.99s
```

Figure 8.2: Terminal Output of own modules php artisan test

Appendix B: GitHub Repository URL

Team Repository: <https://github.com/NickFoo0924/edusystem>

Branch: master